Software Testing and PyTest

Ryan M. Richard

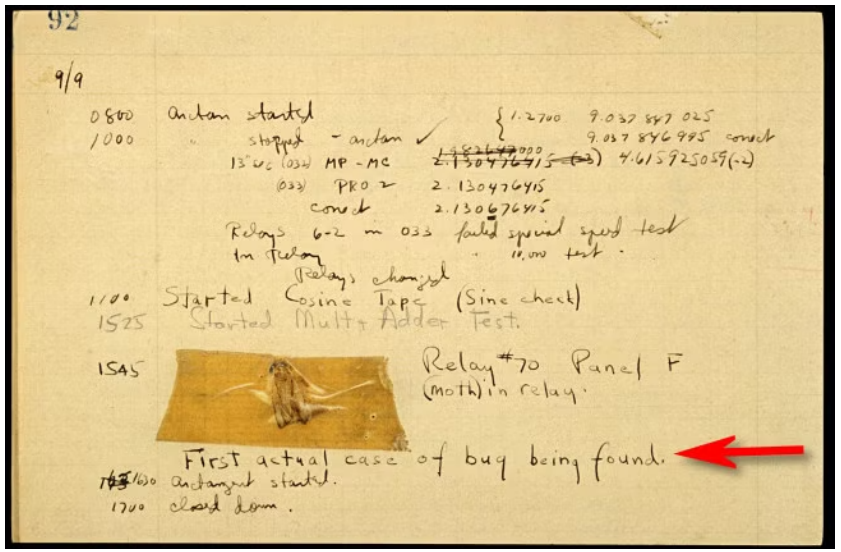
Scientist and Adjunct Assistant Professor of Chemistry
Ames National Laboratory and Iowa State University
Ames, IA, USA

2026 SIMCODES Bootcamp

Terminology

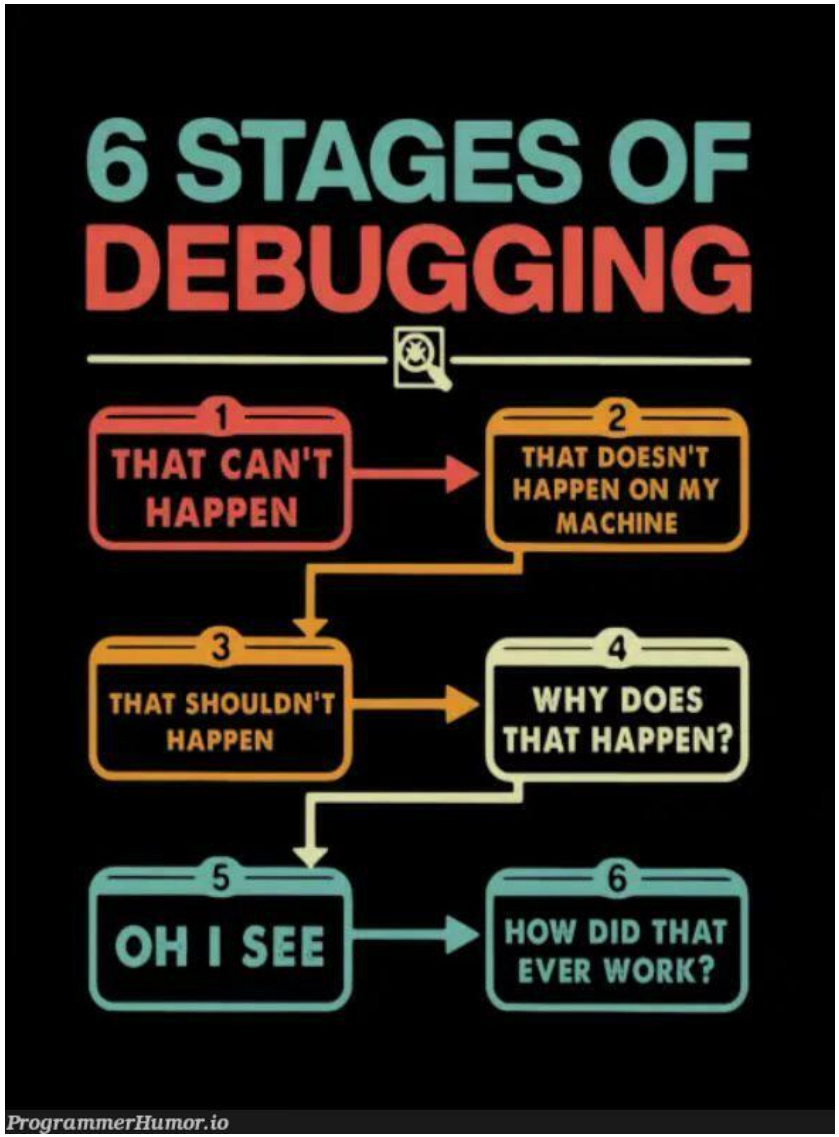
Terminology

- **Bug**. When software behaves unexpectedly.
- Term origin:
  - Apocryphal: Grace Hopper found a moth preventing her program from running.
  - Truth: It's actually older, e.g., Thomas Edison used it to refer to his inventions failing.



Terminology

- **Bug**. When software behaves unexpectedly.
- Term origin:
  - Apocryphal: Grace Hopper found a moth preventing her program from running.
  - Truth: It's actually older, e.g., Thomas Edison used it to refer to his inventions failing.
- **Debugging**. Act of finding bugs.

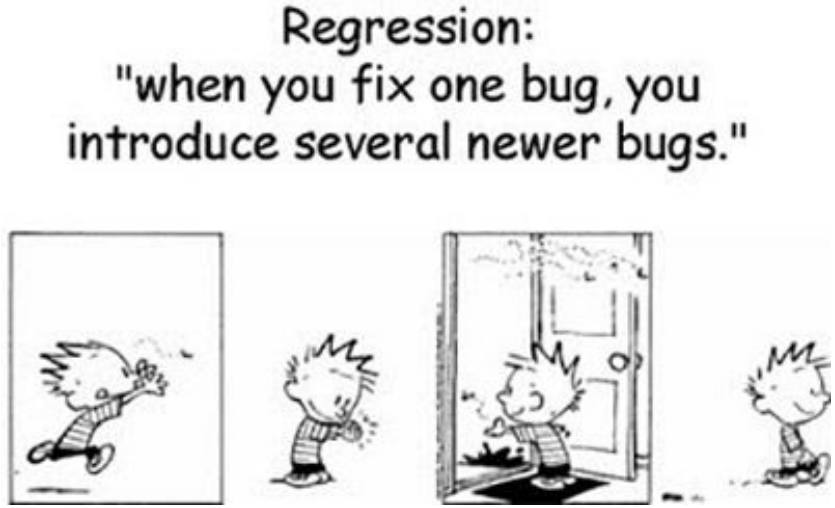


Terminology

- **Test Case**. The *environment*, inputs, outputs, needed to verify correctness.
  - Usually check one thing, and only one thing.
  - Want to check two things? Use two test cases!
- **Test Suite**. A collection of test cases.
- **Testing Framework**. Software for automatically managing test suites.

Terminology

- **Regression**. When a change to working software breaks it.
- **Regression Testing**. Running a test suite to make sure changes do not break the software.



Types of Tests Cases

- **Unit tests**: tests a single function or class in isolation.
- **Integration tests**: tests that multiple components work together.
- **End-to-end tests**: tests the full system from a user's perspective.
- This tutorial focuses on **unit testing**.

Getting Started with PyTest

Motivation

- Software bugs can be costly or even dangerous.
- Manual testing does not scale to large projects.
- Automated tests let you:
  - Catch bugs early and cheaply.
  - Detect regressions when you modify code.
  - Document expected behavior for collaborators.
  - Develop with confidence.
- Rule of thumb: the earlier a bug is found, the easier to fix.

What is PyTest?

- Usually also provide features to manage the test suite.
- Many Python testing frameworks.
- IMHO PyTest is the easiest.
- Will get far just using `assert` statements.

Installing PyTest

- Install once per Python environment.
- **Recommended:** install inside your virtual or conda environment.
- You can also run PyTest as a module: `python -m pytest`

Install with `pip` (standard Python):

```
pip install pytest
```

Install with `conda` (Anaconda/miniconda):

```
conda install pytest
```

Verify the installation:

```
pytest --version
# pytest 8.x.y
```

Test Discovery: PyTest's Naming Rules

- PyTest **automatically finds** tests — no configuration needed.
- It searches the current directory (and subdirectories) for:
  - Files named `test_*.py` or `*_test.py`.
  - Functions named `test_*()` inside those files.
  - Methods named `test_*()` inside `Test*` classes.
- Follow these conventions and PyTest finds everything automatically.

```
# Discovered: file is named test_math.py

# Discovered: starts with test_
def test_add():
    ...

# NOT discovered: missing test_ prefix
def check_add():
    ...

# NOT discovered: too generic
def test():
    ...
```

Running Tests

- Run all tests in the current directory tree.
- Verbose output (show each test name).
- Run tests in a specific file.
- Run a single test by name.
- Run tests matching a keyword.
- Stop after the first failure.

```
pytest

pytest -v

pytest test_math_utils.py

pytest test_math_utils.py::test_add

pytest -k "add"

pytest -x
```

Writing Your First Test

Example Module: `math_utils.py`

Create a file `math_utils.py`. We will use these three functions as running examples.

```python
import math

def add(a, b):
    """Return the sum of a and b."""
    return a + b

def divide(a, b):
    """Return a divided by b. Raises ValueError if b is zero."""
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b

def circle_area(radius):
    """Return the area of a circle with the given radius."""
    if radius < 0:
        raise ValueError("Radius cannot be negative")
    return math.pi * radius ** 2
```

Your First Test File: `test_math_utils.py`

```python
# test_math_utils.py
from math_utils import add

def test_add_positive_numbers():
    assert add(2, 3) == 5

def test_add_negative_numbers():
    assert add(-1, -1) == -2

def test_add_with_zero():
    assert add(0, 5) == 5
```

- Import the function you want to test.
- Each test function must start with `test_`.
- Use Python's built-in `assert` statement.
- One logical check per test is best practice.
- Descriptive names act as documentation.

Run the tests:

```
pytest -v test_math_utils.py
```

Reading PyTest Output

All tests pass

```
test_math_utils.py::test_add_positive
     PASSED
test_math_utils.py::test_add_negative
     PASSED
test_math_utils.py::test_add_with_zero
     PASSED

3 passed in 0.01s
```

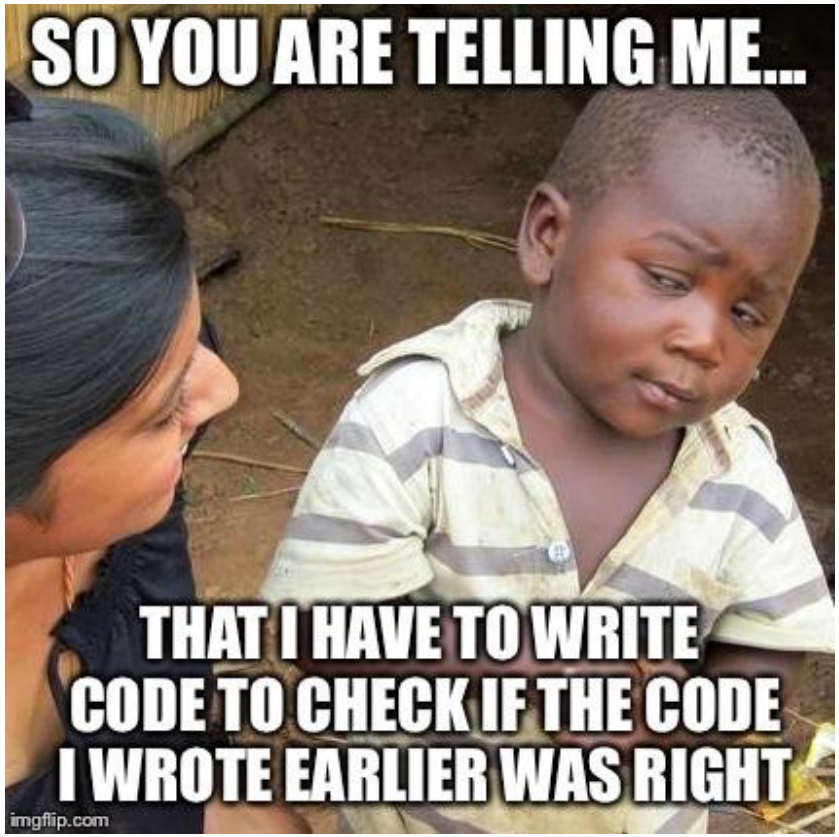
A failing test

```
FAILED test_math_utils.py::
     test_add_positive
AssertionError: assert 4 == 5
  where 4 = add(2, 3)

1 failed in 0.02s
```

- `.` = passed `F` = failed
- `E` = error `s` = skipped
- PyTest shows exactly what went wrong and where.

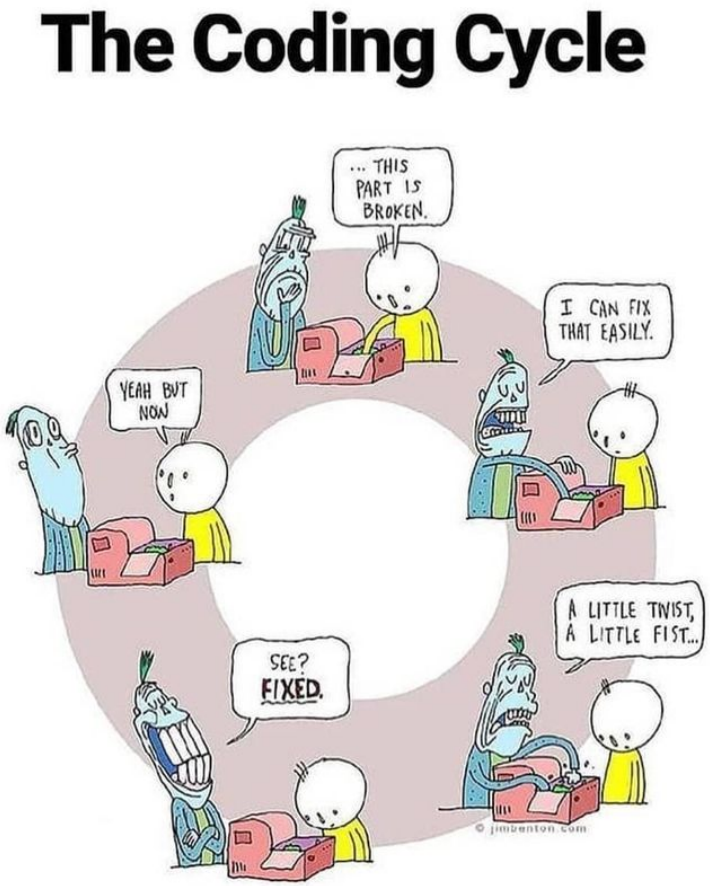
Hands-On: Write Your First Test

- Create `math_utils.py` with the three functions from the example.
- Create `test_math_utils.py`.
- Write at least one test for `add()`.
- Run `pytest -v` and confirm all tests pass.
- Deliberately break a test (e.g., `assert add(2,3) == 99`) and observe the output.
- Fix it and re-run.



Hands-On: Exploring PyTest Options

- Experiment with these flags:
  - `pytest -v` — verbose mode
  - `pytest -x` — stop on first failure
  - `pytest -k "zero"` — run tests matching keyword
- Goal: get comfortable with the edit-test-fix cycle.



The Floating Point Problem

- Floating point arithmetic is not exact.
- Never use == to compare two floats in tests.
- Use `pytest.approx` instead.
- Default tolerance: 10^{-6} (relative).
- Can set absolute or relative tolerance.

```python
import pytest, math
from math_utils import circle_area

# BAD: may fail due to floating point
def test_area_bad():
    assert circle_area(1.0) == math.pi

# GOOD: use pytest.approx
def test_area():
    assert circle_area(1.0) == pytest.approx(math.pi)

# Custom absolute tolerance
def test_area_loose():
    assert circle_area(1.0) == pytest.approx(math.pi,
                                             abs=1e-4)
```

Testing That Exceptions Are Raised

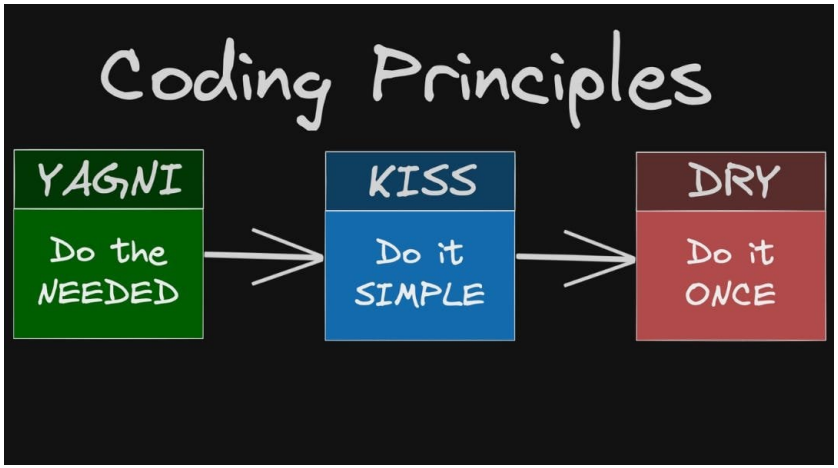
```python
import pytest
from math_utils import divide, circle_area

def test_divide_by_zero():
    with pytest.raises(ValueError):
        divide(10, 0)

def test_divide_error_message():
    with pytest.raises(ValueError,
                       match="Cannot divide by zero"):
        divide(10, 0)

def test_negative_radius():
    with pytest.raises(ValueError):
        circle_area(-1.0)

def test_divide_does_not_raise():
    # No exception should be raised for valid input
    result = divide(10, 2)
    assert result == pytest.approx(5.0)
```

- Use `pytest.raises()`.
- The test **passes** if the expected exception is raised.
- The test **fails** if the exception is NOT raised.
- The `match` argument checks the error message (regex).
- Test that your code fails gracefully, not just that it succeeds!

Fixtures

What Are Fixtures?

- A **fixture** is reusable setup code shared across tests.
- Common uses:
  - Saves time/code creating objects that multiple tests need.
  - Opening and closing files.
  - Establishing a known state before each test.
  - Setup/teardown that needs to occur.
- Avoids copy-paste of setup code.
- Reduces the code needed to write tests.



Using Fixtures

```python
import pytest
from math_utils import divide

@pytest.fixture
def numerator():
    return 10.0

@pytest.fixture
def denominator():
    return 2.0

def test_divide(numerator, denominator):
    assert divide(numerator, denominator)
     == 5.0

def test_divide_type(numerator, denominator
    ):
    result = divide(numerator, denominator)
    assert isinstance(result, float)
```

- Decorate a function with `@pytest.fixture`.
- Add the fixture name as a test parameter.
- PyTest injects the return value automatically.
- Multiple tests can share the same fixture.

Fixtures with Setup and Teardown

```python
import pytest, tempfile, os

@pytest.fixture
def temp_file():
    # Setup: runs before the test
    fd, path = tempfile.mkstemp()
    os.close(fd)

    yield path        # value provided to test

    # Teardown: always runs after the test
    if os.path.exists(path):
        os.remove(path)

def test_file_is_created(temp_file):
    assert os.path.exists(temp_file)

def test_file_is_writable(temp_file):
    with open(temp_file, 'w') as f:
        f.write("hello")
    assert os.path.getsize(temp_file) > 0
```

- Use `yield` instead of `return` to add teardown.
- Code *before* `yield` = setup.
- Code *after* `yield` = teardown (always runs, even if the test fails).
- Ideal for resources that need cleanup, e.g. files.

Misc Features

Testing Many Cases: `@pytest.mark.parametrize`

```python
import pytest
from math_utils import add

@pytest.mark.parametrize("a, b, expected",
      [
    (1,    2,    3),
    (0,    0,    0),
    (-1,   1,    0),
    (-2,  -3,   -5),
    (100, 200,  300),
])
def test_add(a, b, expected):
    assert add(a, b) == expected
```

Running produces:

```
test_add[1-2-3]          PASSED
test_add[0-0-0]          PASSED
test_add[-1-1-0]         PASSED
test_add[-2--3--5]       PASSED
test_add[100-200-300]    PASSED
```

- Runs the test once per parameter set.
- First argument: comma-separated parameter names (as a string).
- Second argument: list of tuples, one per test case.
- PyTest generates a unique ID for each case.
- Equivalent to 5 separate test functions!
- Failures pinpoint exactly which inputs caused the problem.

conftest.py File

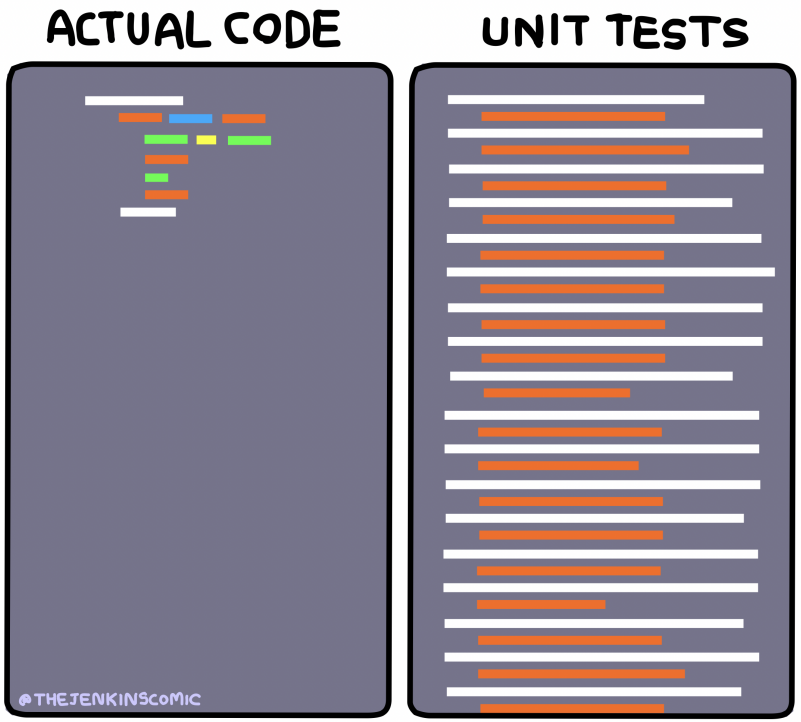
- *conftest.py* is automatically loaded by PyTest.
- Used to store common functionality needed throughout tests.
- Advanced features: fixture scope, pytest plugins, hooks

```python
import pytest

@pytest.fixture(scope="function")
def sample_data():
    """Fixture that provides sample data for tests."""
    return {"numbers": [1, 2, 3], "text": "pytest"}
```

Summary

What We Covered

- **Why test**: catch bugs early, prevent regressions, document expected behavior.
- **Installation**: `pip install pytest` or `conda install pytest` .
- **Discovery**: files named `test_*.py` , functions named `test_*` .
- **Running**: `pytest -v` , `-k` , `-x` .
- **Assertions**: use `assert` ; use `pytest.approx` for floats.



Acknowledgements

- Claude for a first draft.
- PyTest documentation (`docs.pytest.org`).
- NSF for funding the REU.
- ISU and Ames National Lab